

데이터 구조

Chapter 2. Arrays and Structures



고려대학교
전기전자전파공학부
최 린



高麗大學校

Computer System Laboratory

Array as an ADT

- Array

- Programmers view an array as “a consecutive set of memory locations”
 - Usually implemented as a consecutive set of memory locations
 - 자료구조와 그 기억장소 구현의 혼동
- An array is a set of pairs <index, value>
 - Each index has a value associated with it
 - Mathematically, this is called a mapping between an index and a value

$$array : i \mapsto a_i$$

- 배열이 반드시 일련의 연속적인 메모리 위치에 저장될 필요는 없음



ADT Array

ADT Array

objects : index의 각 값에 대하여 집합 item에 속한 한 값이 존재하는 $\langle \text{index}, \text{value} \rangle$ 쌍의 집합.

index는 일차원 또는 다차원 유한 순서 집합이다.

예를 들면, 1차원의 경우 $\{0, \dots, n-1\}$ 과

2차원 배열 $\{(0,0), (0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1), (2,2)\}$ 등.

functions : 모든 $A \in \text{Array}$, $i \in \text{index}$, $x \in \text{item}$, j , $\text{size} \in \text{integer}$

Array Create(j , list) ::= return an array of j dimensions

Item Retrieve(A , i) ::= if ($i \in \text{index}$)

return the item associated with index value i
in array A .

else return error.

Array Store(A , i , x) ::= if ($i \in \text{index}$)

return 새로운 쌍 $\langle i, x \rangle$ 가 삽입된 배열 A .

else return error.

end Array

< Array 추상 데이터 타입 >



Arrays in C (1/3)

- C의 일차원 배열

int list[5], *plist[5];

- 정수값 : list[0], list[1], list[2], list[3], list[4]
- 정수포인터 : plist[0], plist[1], plist[2], plist[3], plist[4]

변 수	메모리 주소
List[0]	기본주소 = a
List[1]	a + sizeof(int)
List[2]	a + 2 · sizeof(int)
List[3]	a + 3 · sizeof(int)
List[4]	a + 4 · sizeof(int)

- 포인터 해석

int *list1; int list2[5];

- list2 = list2[0]를 가리키는 포인터
- list2 + i = list2[i]를 가리키는 포인터, 타입의 크기와 상관 없음
- (list2+i) = &list2[i], *(list2+i) = list2[i]



Arrays in C (2/3)

- 배열 프로그램의 예

```
#define MAX_SIZE 100
float sum(float [], int);
float input[MAX_SIZE], answer;
int i;
void main(void)
{
    for(i=0; i<MAX_SIZE; i++)
        input[i] = i;
    answer = sum(input, MAX_SIZE);
    printf("The sum is: %f\n", answer);
}
float sum(float list[], int n)
{
    int i;
    float tempsum = 0;
    for(i = 0; i<n; i++)
        tempsum += list[i];
    return tempsum;
}
```

- 호출 시 input(&input[0])
이 formal parameter *list*에
복사
 - C 언어의 parameter 전달방
식: **call-by-value**
 - Call-by-value 임에도 불구하
고 array의 주소가 parameter
로 전달되기 때문에 배열
의 값들을 변경할 수 있다.



Arrays in C (3/3)

- 예제[일차원 배열의 주소 계산]

```
int one[] = {0, 1, 2, 3, 4};  
print1(&one[0], 5)
```

```
void print1 (int *ptr, int rows)  
{/* print out a one-dimensional array using a pointer */  
  int i;  
  printf ("Address Contents\n");  
  for (i=0; i<rows; i++)  
    printf ("%8u%5d\n", ptr + i, *(ptr + i));  
  printf ("\n");  
}
```

주소	1228	1232	1236	1240	1244
내용	0	1	2	3	4

< 1차원 배열의 주소 계산 >



Dynamically Allocated Arrays (1/4)

- 이전 프로그램 예제에서는 MAX_SIZE를 사용하여 배열의 크기를 결정
 - 배열이 더 큰 경우는 MAX_SIZE를 변경하여 재 컴파일 수행
 - MAX_SIZE를 큰 수로 설정하는 경우 메모리 부족 문제 발생
- 동적 할당: 메모리 크기에 대한 결정을 실행 시간으로 연기

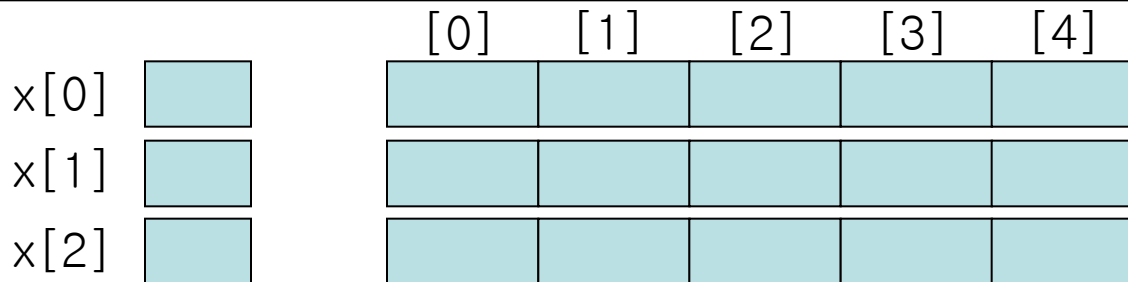
```
int i, n, *list;
printf("Enter the number of numbers to generate: ");
scanf("%d", &n);
If (n < 1) {
    fprintf(stderr, "Improper value of n \n");
    exit (EXIT_FAILURE);
}
MALLOC(list, n * sizeof(int));
```

※ N<1이거나 정렬할 수의 리스트를 저장 할 메모리가 충분치 않을때 실패



Dynamically Allocated Arrays (2/4)

- 2차원 배열
 - `int x[3][5]`



```
int ** make2dArray(int rows, int cols)
{ /* create a two dimensional rows × cols array */
    int **x, i;

    /* get memory for row pointers */
    MALLOC (x, rows * sizeof (*x));

    /* get memory for each row */
    for (i = 0; i < rows; i++)
        MALLOC(x[i], cols * sizeof(**x));
    return x;
}
```

< 2차원 배열의 동적 생성 >



Dynamically Allocated Arrays (3/4)

• Ex>

```
int **myArray;  
myArray = make2dArray(5, 10);  
myArray[2][4] = 6
```

- 5×10 2차원 정수 배열에 대한 메모리 할당
- 이 배열의 [2][4]원소에 정수 6을 지정

• Calloc / Realloc 함수

```
int *x;  
x = calloc(n, sizeof(int))
```

- calloc : 사용자가 지정한 양의 메모리를 할당하고
할당된 메모리를 0으로 초기화
- realloc : malloc이나 calloc으로 이미 할당된 메모리 크기
재조정



Dynamically Allocated Arrays (4/4)

```
#define CALLOC(p, n, s)\
    if(!((p) = calloc(n, s))) {\
        fprintf(stderr, "Insufficient memory");\
        exit(EXIT_FAILURE);\
    }
```

```
#define REALLOC(p, s)\
    if(!((p) = realloc(p, s))) {\
        fprintf(stderr, "Insufficient memory");\
        exit(EXIT_FAILURE);\
    }
```



Structures and Union (1/5)

- 구조(structure) : struct
 - Structures are collections of data items of *different* data types
 - Also called *records* in other programming languages
 - In contrast, arrays are collection of data of the *same* type
 - Each item is defined as to its *type* and *name*

```
struct {  
    char name[10];  
    int age;  
    float salary;  
} person;
```

- Member operators

```
strcpy(person.name, "james");  
person.age = 10;  
person.salary = 35000;
```



Structures and Union (2/5)

- typedef 문장을 사용한 Structure *data type* 생성

```
typedef struct human being{  
    char name[10];  
    int age;  
    float salary;  
};
```

or

```
typedef struct {  
    char name[10];  
    int age;  
    float salary;  
} human_being;
```

- 변수선언

```
human_being person1, person2 ;  
    if (strcmp(person1.name, person2.name))  
        printf(“두 사람의 이름은 다르다.\n”);  
    else  
        printf(“두 사람의 이름은 같다.”)
```

- Structure assignment: $person1 = person2$ (ANSI C)
strcpy(person1.name, person2.name); (old version of C)
person1.age = person2.age;
person1.salary = person2.salary;
- 전체 구조의 동등성 검사 : “if ($person1 == person2$)” is not possible



Structures and Union (3/5)

- 구조의 동등성 검사

```
#define FALSE 0  
#define TRUE 1
```

```
int humansEqual (humanBeing person1, humanBeing person 2)  
{/* 만일 person1과 person2가 동일인이면 TRUE를 반환하고 그렇지  
   않으면 FALSE를 반환한다. */  
    if (strcmp(person1.name, person2.name))  
        return FALSE;  
    if (person1.age != person2.age)  
        return FALSE;  
    if (person1.salary != person2.salary)  
        return FALSE;  
    return TRUE;  
}
```

< 구조의 동등성을 검사하는 함수 >



Structures and Union (4/5)

- 유니언

- Union의 필드들은 메모리 공간을 공유
- 한 필드 만 어느 한 시점에 활동적이 되어 사용 가능

```
typedef struct sex_type {  
    enum tagField {female, male} sex;  
    union {  
        int children;  
        int beard;  
    } u;  
};  
  
typedef struct human_being {  
    char name[10];  
    int age;  
    float salary;  
    sex_type sex_info;  
};  
  
human_being person1, person2;
```



Structures and Union (5/5)

– 값 할당

```
person1.sex_info.sex = male;  
person1.sex_info.u.beard = FALSE;  
  
person2.sex_info.sex = female;  
person2.sex_info.u.children = 4;
```

• Implementation of Structures

struct {int i, j; float a, b;} or struct {int i; int j; float a; float b;}

- Values will be stored in increasing memory locations in the order specified in the structure definition
- Holes or padding may actually occur within a structure to permit two consecutive components to be properly aligned within memory. (*memory alignment*)
- The size of an object of a structure or union type is the amount of storage necessary to represent the largest component, including any padding.
- 구조는 같은 메모리 경계에서 시작하고 끝나야 함
 - 짝수바이트거나 4, 8, 16등의 배수가 되는 메모리 경계



Self-referential Structures

- 구성요소중 자신을 가리키는 포인터가 존재하는 구조

```
typedef struct list {  
    char data;  
    list *link;  
};
```

```
list item1, item2, item3;  
item1.data = 'a';  
item2.data = 'b';  
item3.data = 'c';  
item1.link = item2.link = item3.link = NULL;  
  
item1.link = &item2;   구조들을 서로 연결  
item2.link = &item3;   (item1 → item2 → item3)
```



Ordered List

- Arrays can be used to implement other abstract data types
- Ordered list(or linear list)
 - One of the simplest and most commonly used data structures
 - 원소들의 순서가 있는 모임
 - $A = (a_1, a_2, \dots, a_n)$ $a_i : \text{atom}, n=0 : \text{empty list}$
 - 한 주일의 요일들(일, 월, 화, 수, 목, 금, 토)
 - 카드 한 벌의 값(ace, 2, 3, 4, 5, 6, 7, 8, 9, 10, j, q, k)
 - 건물의 층(지하실, 로비, 일층, 이층)
 - 미국의 2차 세계 대전 참전 연도(1941, 1942, 1943, 1944, 1945)
 - 연산
 - 리스트의 길이 n 의 계산
 - 리스트의 항목을 왼쪽에서 오른쪽 (오른쪽에서 왼쪽)으로 읽기
 - 리스트로부터 i 번째 항목을 검색, $0 \leq i \leq n$
 - 리스트의 i 번째 항목을 대체, $0 \leq i \leq n$
 - 리스트의 i 번째 위치에 새로운 항목을 삽입(i 번째 위치, $0 \leq i \leq n$)
 - 리스트의 i 번째 항목을 제거(i 번째 항목, $0 \leq i < n$)



Ordered List

- Implementation of ordered lists
 - 메모리(기억장소) 표현
 - Sequential mapping
 - Represent an ordered list *as an array*
 - Each item is stored in consecutive memory locations
 - 대부분의 연산에 대해서는 잘 작동
 - Insertion and deletion pose problems since the sequential allocation forces us to move items so that the sequential mapping is reserved.
 - Non-sequential mapping
 - Represent an ordered list *as a linked list*
 - 리스트 : pointer(links)를 가지는 노드 집합
 - 비연속적 기억장소 위치



Polynomials

- Polynomial

- An example of an ordered list
- A polynomial is a sum of terms where each term has a form ax^e

$$A(x) = 3x^{20} + 2x^5 + 4, \quad B(x) = x^4 + 10x^3 + 3x^2 + 1$$

$$A(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x^1 + a_0 x^0$$

ax^e a : coefficient $a_n \neq 0$

e : exponent – unique

x : variable x

- 차수(degree) : 다항식에서 가장 큰 지수

- $A(x) = \sum a_i x^i, B(x) = \sum b_i x^i$

$$A(x) + B(x) = \sum (a_i + b_i) x^i$$

$$A(x) \cdot B(x) = \sum (a_i x^i \cdot (\sum b_j x^j))$$



Polynomial ADT

- Polynomial 추상 데이터 타입

Structure Polynomial

Objects : $P(x) = a_1x^{e_1} + \dots + a_nx^{e_n}$: $\langle e_i, a_i \rangle$ 의 순서쌍 집합.

여기서 a_i 는 Coefficient, e_i 는 Exponents. $e_i(\geq 0)$ 는 정수

Function : 모든 $\text{poly}, \text{poly1}, \text{poly2} \in \text{polynomial}, \text{coef} \in \text{Coefficients}$
 $\text{Expon} \in \text{Exponents}$ 에 대해

Polynomial Zero() ::= return 다항식, $p(x) = 0$

Boolean IsZero(poly) ::= if(poly) return FALSE
else return TRUE

Coefficient Coef(poly, expon) ::= if(expon $\in$ poly) return 계수
else return 0

Exponent Lead_Exp(poly) ::= return poly에서 제일 큰 지수

Polynomial Attach(poly, coef, expon) ::= if(expon $\in$ poly) return 오류
else return $\langle \text{coef}, \text{exp} \rangle$ 항이
삽입된 다항식 poly



Polynomial ADT

```
Polynomial Remove(poly, expon) ::= if(expon  $\in$  poly)
                                return 지수가 expon인 항이
                                삭제된 다항식 poly
                                else return 오류
Polynomial SingleMult(poly, coef, expon) ::= return 다항식  $\text{poly} \cdot \text{coef} \cdot x^{\text{expon}}$ 
Polynomial Add(poly1, poly2) ::= return 다항식  $\text{poly1} + \text{poly2}$ 
Polynomial Mult(poly1, poly2) ::= return 다항식  $\text{poly1} \cdot \text{poly2}$ 
```

end polynomial



Polynomial Representation

- 다항식의 표현 (I)

- Store the coefficients in order of decreasing exponents

```
#define MAX_DEGREE 101    /* 다항식의 최대 차수 +1*/
typedef struct {
    int degree;
    float coef[MAX_DEGREE];
} polynomial;
```

- A : polynomial

- $A(x) = \sum a_i x^i$ a.degree = n, a.coef[i] = a_{n-i} , $0 \leq i \leq n$

※ $A = (n, \underline{a_n}, \underline{a_{n-1}}, \dots, \underline{a_1}, \underline{a_0})$

degree of A n+1 coefficients

예) $A(x) = x^4 + 10x^3 + 3x^2 + 1$: n = 4
 $A = (4, 1, 10, 3, 0, 1)$: 6 elements



Polynomial Representation

- 함수 padd의 초기 버전

```
/* d = a + b, 여기서 a, b, d는 다항식이다. */  
d = Zero();  
while (! IsZero(a) && ! IsZero(b)) do {  
    switch COMPARE(Lead_Exp(a), Lead_Exp(b)) {  
        case -1:  
            d = Attach (d, Coef(b, Lead_Exp(b)), Lead_Exp(b));  
            b = Remove(b, Lead_Exp(b));  
            break;  
        case 0:  
            sum = Coef(a, Lead_Exp(a)) + Coef(b, Lead_Exp(b));  
            if (sum) {  
                Attach(d, sum, Lead_Exp(a));  
                a = Remove(a, Lead_Exp(a));  
                b = Remove(b, Lead_Exp(b));  
            }  
            break;  
    }
```



Polynomial Representation

```
case 1;  
    d = Attach(d, Coef(a, Lead_Exp(a)), Lead_Exp(a));  
    a = Remove(a, Lead_Exp(a));  
}  
}
```

a 또는 b의 나머지 항을 d에 삽입한다.

- 문제점

- $a.degree \ll MAX_DEGREE$

- ⇒ $a.coef[MAX_DEGREE]$ 의 대부분이 필요없음



Polynomial Representation

- 다항식의 표현 (II): 계수와 지수의 쌍으로 표현

```
MAX_TERMS 100    /* 항 배열의 크기*/  
typedef struct {  
    float coef;  
    int expon;  
} polynomial;  
polynomial terms[MAX_TERMS];  
int avail = 0;
```

- (+) Solves the problem of many nonzero terms
- (-) When all the terms are nonzero, this representation requires about twice as much space as the first one.



Polynomial Representation

- Example: $A(x) = 2x^{1000} + 1$, $B(x) = x^4 + 10x^3 + 3x^2 + 1$
 - These polynomials are stored in the array terms
 - The index of the first term of A and B is given by $startA$ and $startB$.
 - The index of the last term of A and B is given by $finishA$ and $finishB$.
 - $avail$ is the index of the next free location

	$startA$	$finishA$	$startB$		$finishB$	$avail$
	↓	↓	↓		↓	↓
<i>coef</i>	2	1	1	10	3	1
<i>exp</i>	1000	0	4	3	2	0
	0	1	2	3	4	5
						6

- We can place many polynomials in the array terms
- To use $A(x)$, we must pass in $startA$ and $finishA$



Polynomial Addition ($D=A+B$)(1/2)

```
void padd(int starta, int finisha, int startb, int finishb, int *startd, int *finishd);
{ /* A(x)와 B(x)를 더하여 D(x)를 생성한다. */
  float coefficient;
  *startd = avail;
  while (starta <= finisha && startb <= finishb)
    switch (COMPARE(terms[starta].expon, term[startb].expon)) {
      case -1: /* a의 expon이 b의 expon보다 작은 경우 */
        attach(terms[startb].coef, terms[startb].expon);
        startb++; break;
      case 0: /* 지수가 같은 경우 */
        coefficient = terms[starta].coef + terms[startb].coef;
        if(coefficient) attach(coefficient, terms[starta].expon);
        starta++; startb++; break;
      case 1: /* a의 expon이 b의 expon보다 큰 경우 */
        attach(terms[starta].coef, terms[starta].expon);
        starta++;
    }
}
```



Polynomial Addition(2/2)

```
/* A(x)의 나머지 항들을 첨가한다. */  
for(; starta <= finisha; starta++)  
    attach(terms[starta].coef, term[starta].expon);  
/* B(x)의 나머지 항들을 첨가한다. */  
for(; startb <= finishb; startb++)  
    attach(terms[startb].coef, terms[startb].expon);  
*finishd = avail - 1;
```

< 두 다항식을 더하는 함수 >

```
void attach(float coefficient, int exponent)  
{  
    /* 새 항을 다항식에 첨가한다. */  
    if (avail >= MAX_TERMS) {  
        fprintf(stderr, “다항식에 항이 너무 많다.”);  
        exit(1);    }  
    terms[avail].coef = coefficient;  
    terms[avail++].expon = exponent;  
}
```

< 새로운 항을 첨가하는 함수 >



Analysis of PADD

- $m, n(>0)$: the number of nonzero terms in A and B respectively
 - while 루프
 - 각 반복마다 starta나 startb 또는 둘 다 값이 증가
 - 반복 종료 $\rightarrow$ starta $\leq$ finisha && startb $\leq$ finishb
 - 반복 횟수 $\leq m+n-1$, i.e. $O(m+n)$
 - Worst case: $A(x) = \sum_{i=0}^n x^{2i}$ 와 $\sum_{i=0}^n x^{2i+1}$
 - for 루프 : $O(m+n)$
- $\therefore$ 연산시간 = **$O(m+n)$**



Sparse Matrix (1/3)

- $m \times n$ matrix $A \equiv A[MAX_ROWS][MAX_COLS]$

$$\frac{\text{no. of non-zero elements}}{\text{no. of total elements}} \ll \text{small}$$

- 희소 행렬

	col0	col1	col2
row0	-27	3	4
row1	6	82	-2
row2	109	-64	11
row3	12	8	9
row4	48	27	47

	col0	col1	col2	col3	col4	col5
row0	15	0	0	22	0	-15
row1	0	11	3	0	0	0
row2	0	0	0	-6	0	0
row3	0	0	0	0	0	0
row4	91	0	0	0	0	0
row5	0	0	28	0	0	0



Sparse Matrix(2/3)

- ADT SparseMatrix

ADT SparseMatrix

object: 3원소 쌍 <행, 열, 값>의 집합이다. 여기서, 행과 열은 정수이고 이 조합은 유일하며, 값은 item 집합의 원소이다.

functions : 모든 $a, b \in \text{sparseMatrix}$, $x \in \text{item}$, i, j , maxCol , $\text{maxRow} \in \text{index}$ 에

$\text{sparseMatrix Create}(\text{maxRow}, \text{maxCol}) ::=$

return maxItems까지 저장할 수 있는 sparseMatrix
여기서 최대 행의 수는 maxRow이고
최대 열의 수는 maxCol이라 할 때
 $\text{maxItems} = \text{maxRow} \times \text{maxCol}$ 이다.

$\text{sparseMatrix Transpose}(a) ::=$

return 모든 3원소 쌍의 행과 열의 값을 교환하여
얻은 행렬

$\text{sparseMatrix Add}(a, b) ::=$

if a와 b의 차원이 같으면
return 대응 항들 즉, 똑같은 행과 열의 값을
가진 항들을 더해서 만들어진 행렬
else return 에러



Sparse Matrix(3/3)

sparseMatrix Multiply(a,b) ::=

if a의 열의 수와 b의 행의 수가 같으면
return 다음 공식에 따라 a와 b를 곱해서 생성된
행렬 $d : d(i, j) = \sum (a[i][k] b[k][j])$
여기서 $d(i, j)$ 는 (i, j) 번째 원소이다.
else return 에러

- Sparse matrix representation
 - Represented by an array of triples <row, col, value>
 - Organize the triples so that the row indices are in ascending order
 - Also, for each row, column indices are in ascending order
 - For an array A, A[0].row contains the number of rows, A[0].col contains the number of columns, and A[0].val contains the total number of nonzero entries
 - 연산 종료를 보장하기 위해 행과 열의 수와 행렬 내에 있는 0이 아닌 항의 수를 알아야 함



Matrix Transpose (1/3)

```
#define MAX_TERMS 101 /* maximum number of terms + 1 */
typedef struct {
    int col;
    int row;
    int value;
} term;
Term a[MAX_TERMS];
```

	행	열	값
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

	행	열	값
b[0]	6	6	8
b[1]	0	0	15
b[2]	0	4	91
b[3]	1	1	11
b[4]	2	1	3
b[5]	2	5	28
b[6]	3	0	22
b[7]	3	2	-6
b[8]	5	0	-15



Matrix Transpose(2/3)

- Matrix Transpose

```
for (j = 0; j < columns; j++)  
    for(i = 0; i < rows; i++)  
        b[j][i] = a[i][j];
```

```
void transpose(term a[], term b[])    /* a를 전치시켜 b를 생성 */  
{  
    int n, i, j, currentb;  
    n = a[0].value;                  /* 총 원소 수 */  
    b[0].row = a[0].col;              /* b의 행 수 = a의 열 수 */  
    b[0].col = a[0].row;              /* b의 열 수 = a의 행 수 */  
    b[0].value = n;  
    if (n>0) {                       /* 0이 아닌 행렬 */  
        currentb = 1;  
        for (i=0; i=a[0].col; i++)    /* a에서 열별로 전치 */  
            for (j=1; j<n; j++)        /* 현재의 열로부터 원소를 찾는다 */  
                if (a[j].col == i {    /* 현재의 열에 있는 원소를 b에 첨가한다. */  
                    b[currentb].row = a[j].col;  
                    b[currentb].col = a[j].row;  
                    b[currentb].value = a[j].value;  
                    currentb++; }  
    }  
}
```

*What is the time complexity
of this algorithm?*



Matrix Transpose(3/3)

- Fast Transpose

```
void fastTranspose(term a[], term b[])    /* a를 전치시켜 b를 생성 */
{
    int rowTerms[MAX_COL], startingPos[MAX_COL];
    int i, j, numCol = a[0].col, numTerms = a[0].value;
    b[0].row = numCols; b[0].col = a[0].row;
    b[0].value = numTerms;
    if (numTerms > 0) { /* 0이 아닌 행렬 */
        for(i=0; i<numCols; i++)
            rowTerms[i] = 0;
        for(i=1; i<=numTerms; i++)          /* First determine the number of elements */
            rowTerms[a[i].col]++;           /* in each column */
        startingPos[0]=1;
        for(i=1; i<numCols; i++)            /* Determine the start position of each row */
            startingPos[i] =                /* in the transpose matrix */
                startingPos[i-1] + rowTerms[i-1];
        for(i=1; i<=numTerms; i++) {
            j=startingPos[s[i].col]++;
            b[j].row = a[i].col; b[j].col = a[i].row;
            b[j].value = a[i].value; }
    }
}
```

*What is the time complexity
of this algorithm?*



Matrix Multiplication (1/3)

- 정의

- $m \times n$ 행렬 A와 $n \times p$ 행렬 B가 주어질 때 곱셈 결과 행렬인 D는 $m \times p$ 차원을 가지며, $0 \leq i \leq m, 0 \leq j \leq p$ 에 대해 원소 $\langle i, j \rangle$ 는 다음과 같다.

$$d_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$$

1 0 0	1 1 1		1 1 1
1 0 0	0 0 0	=	1 1 1
1 0 0	0 0 0		1 1 1

< 두 희소 행렬의 곱셈 >

⇒ 두 희소행렬의 곱셈 결과는 희소행렬이 아니다



Matrix Multiplication(2/3)

- 희소 행렬의 곱셈

```
void mmult (term a[], term b[], term d[]) { /* 두 희소 행렬을 곱한다. */
int i, j, column, total B = b[0].value, totalD = 0;
int rowsA = a[0].row, colsA = a[0].col;
totalA = a[0].value; int colsB = b[0].col;
int rowBegin = 1, row = a[1].row, sum=0;
int rowB[MAX_TERMS][3];
If (colsA != b[0].row) {
    fprintf(stderr, "Incompatible matrices\n");
    exit(1);
}
fastTranspose(b, newB); /* 경계 조건 설정 */
a[totalA+1].row = rowA;
newB[totalB+1].row = colsB;
newB[totalB+1].col = 0;
    column = newB[1].row;
    for (j=1; j<= totalB+1) { /* a의 행과 b의 열을 곱한다. */
        if (a[i].row != row) {
            storeSum(d, &totalD, row, column, &sum);
            i = rowBegin;
            for (; newB[j].row == column; j++)
                column = newB[j].row;
```



Matrix Multiplication(3/3)

```
}
else if (newB[j].row != column) {
    storeSum(d, &totalD, row, column, &sum);
    i = rowBegin;
    column = newB[j].row;
}
else if (newB[j].row != column) {
    storeSum(d, &totalD, row, column, &sum);
    i = rowBegin;
    column = newB[j].row;
}
else switch (COMPARE(a[i].col, newB[j].col)) {
    case -1:    /*a의 다음항으로 이동 */
        i++; break;
    case 0:    /*항을 더하고, a와 b를 다음 항으로 이동 */
        sum += (a[i++].value * newb[j++].value);
        break;
    case 1:    /* b의 다음 항으로 이동 */
        j++;
}
} /* for j <= totalB + 1문의 끝 */
for (; a[i].row == row; i++)
    rowBegin = i; row = a[i].row;
} /* for i <= totalA문의 끝 */
d[0].row = rowsA;
d[0].col = colsB; d[0].value = totalD;
}
```



Multidimensional Arrays (1/3)

- 배열의 원소 수 : $a[upper_0][upper_1] \dots [upper_{n-1}]$

$$= \prod_{i=0}^{n-1} upper_i$$

- $a[10][10][10] \Rightarrow 10 \cdot 10 \cdot 10 = 1000$

- 다차원 배열 표현 방법

- 행 우선 순서(row major order)

$A[upper_0][upper_1]$

$A[0][0]$	$A[0][1]$	...	$A[0][upper_1-1]$				
row ₀				row ₁	row ₂	row _{upper0-1}	

- 열 우선 순위(column major order)

$A[0][0]$	$A[1][0]$	...	$A[upper_0-1][0]$				
col ₀				col ₁	col ₂	col _{upper1-1}	



Multidimensional Arrays(2/3)

- 주소 계산 (2차원 배열 - 행우선 표현)
 - $\alpha = A[0][0]$ 의 주소
 - $A[i][0]$ 의 주소 = $\alpha + i \cdot upper_1$
 - $A[i][j]$ 의 주소 = $\alpha + i \cdot upper_1 + j$
- 주소 계산 (3차원 배열) : $A[upper_0][upper_1][upper_2]$
 - 2차원 배열 $upper_1 \times upper_2 \Rightarrow upper_0$ 개
 - $A[i][0][0]$ 주소 = $\alpha + i \cdot upper_1 \cdot upper_2$
 - $A[i][j][k]$ 주소 = $\alpha + i \cdot upper_1 \cdot upper_2 + j \cdot upper_2 + k$
- $A[upper_0][upper_1] \dots [upper_{n-1}]$
 - $\alpha = A[0][0] \dots [0]$ 의 주소
 - $a[i_0][0] \dots [0]$ 주소 = $\alpha + i_0 upper_1 upper_2 \dots upper_{n-1}$
 - $a[i_0][i_1][0] \dots [0]$ 주소
= $\alpha + i_0 upper_1 upper_2 \dots upper_{n-1} + i_1 upper_2 upper_3 \dots upper_{n-1}$



Multidimensional Arrays(3/3)

– $a[i_0][i_1]\dots[i_{n-1}]$ 주소 =

$$\begin{aligned} & \alpha + i_0 upper_1 upper_2 \dots upper_{n-1} \\ & + i_1 upper_2 upper_3 \dots upper_{n-1} \\ & + i_2 upper_3 upper_4 \dots upper_{n-1} \\ & \vdots \\ & + i_{n-2} upper_{n-1} \\ & + i_{n-1} \end{aligned}$$

= $\alpha +$

여기서 :

$$\sum_{j=0}^{n-1} i_j a_j$$

$$\left\{ \begin{array}{l} a_j = \prod_{k=j+1}^{n-1} upper_k \quad 0 \leq j < n-1 \\ a_{n-1} = 1 \end{array} \right.$$



String (1/4)

- 추상 데이터 타입

ADT String

object : 0개 이상의 문자들의 유한 집합

function : 모든 $s, t \in \text{string}$, $i, j, m \in \text{음이 아닌 정수}$

string Null(m) ::= 최대 길이가 m 인 스트링을 반환

초기는 NULL로 설정되며, NULL은 “”로 표현된다.

integer Compare(s, t) ::= if (s 와 t 가 같으면) return 0

else if (s 가 t 에 선행하면) return -1

else return +1

Boolean IsNull(s) ::= if (Compare(s , NULL)) return FALSE

else return TRUE

Integer Length(s) ::= if (Compare(s , NULL)) s 의 문자 수를 반환

else return 0

string Concat(s, t) ::= if (Compare(s , NULL)) s 뒤에 t 를 붙인 스트링을 반환

else return s

string Substr(s, i, j) ::= if ($(j > 0) \ \&\& \ (i + j - 1) < \text{Length}(s)$)

s 에서 $i, i+1, \dots, i+j-1$ 의 위치에 있는 스트링을 반환

else return NULL



String (2/4)

- C언어에서 스트링
 - 널 문자 \0으로 끝나는 문자 배열을 의미

```
#define MAX_SIZE 100      /* 스트링의 최대 크기 */  
char s[MAX_SIZE]={"dog"};  
char t[MAX_SIZE]={"house"};
```



```
char s[]={"dog"};  
char t[]={"house"};
```

s[0]	s[1]	s[2]	s[3]
d	o	g	\0

t[0]	t[1]	t[2]	t[3]	t[4]	t[5]
h	o	u	s	e	\0

< C언어에서 스트링 표현 >



String (3/4)

- 스트링 삽입의 예

s →

a	m	o	b	i	l	e	\0
---	---	---	---	---	---	---	----

t →

u	t	o	\0
---	---	---	----

temp →

\0

initially

temp →

a	\0
---	----

(a) after strncpy (temp, s, 1)

temp →

a	u	t	o	\0
---	---	---	---	----

(b) after strcat (temp, t)

temp →

a	u	t	o	m	o	b	i	l	e	\0
---	---	---	---	---	---	---	---	---	---	----

(c) after strcat (temp, (s+i))



String (4/4)

- 스트링 삽입 함수

```
void strnins (char *s, char *t, int i)
{ /* 스트링 s의 i번째 위치에 스트링 t를 삽입 */
    char string[MAX_SIZE], *temp = string;

    if (i < 0 && i > strlen(s) ) {
        fprintf(stderr, "Position is out of bounds \n");
        exit(1);
    }
    if (!strlen(s))
        strcpy(s, t);
    else if (strlen(t)) {
        strncpy (temp, s, i);
        strcat (temp, t);
        strcat (temp, (s + i));
        strcpy (s, temp);
    }
}
```



Pattern Matching (1/7)

- 좀 더 복잡한 스트링 응용

- pat는 string에서 탐색하기 위한 패턴
- pat가 string 내에 있는지 결정하는 가장 쉬운 방법
⇒ 내장 함수 strstr의 사용

```
if (t=strstr(string, pat))  
    printf("The string from strstr is : %\n", t);  
else  
    printf("The pattern was not found with strstr\n");
```

- If pat is not in the string, returns a null pointer.
- Otherwise, t holds a pointer to the start of pat in string
 - The entire string beginning at position t is printed out
- Several different methods for implementing a pattern matching function
 - The easiest but least efficient method sequentially examines each character of the string until it finds the pattern or it reaches the end of the string. $\Rightarrow O(nm)$ where n is the pattern length and m is the string length



Pattern Matching(2/7)

- 패턴의 마지막 문자를 먼저 검사하는 패턴 매칭

```
int nfind(char *string, char *pat)
{ /* 먼저 패턴의 마지막 문자를 매치시켜 본 뒤에, 처음부터 매치시킨다. */
    int i, j, start = 0;
    int lasts = strlen(string) - 1;
    int lastp = strlen(pat) - 1;
    int endmatch = lastp;

    for (i=0; endmatch <= lasts; endmatch++, start++) {
        if (string[endmatch] == pat[lastp])
            for (j=0; i=start; j< lastp &&
                start[i] == pat[j]; i++, j++)
                ;
        if (j == lastp)
            return start; /* 성공 */
    }
    return -1;
}
```

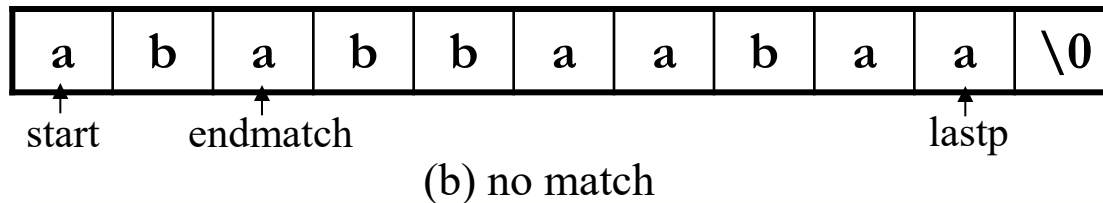
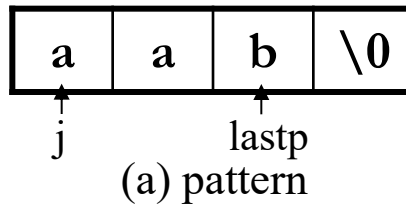
*What is the time complexity
of this algorithm?*

< nfind 함수 >

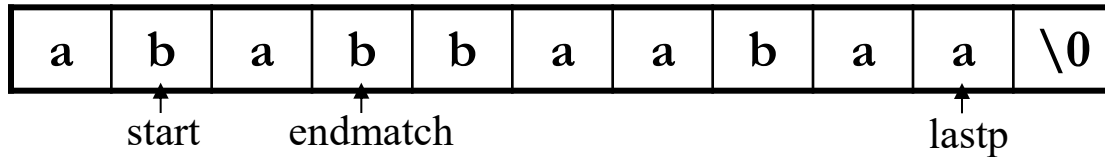


Pattern Matching(3/7)

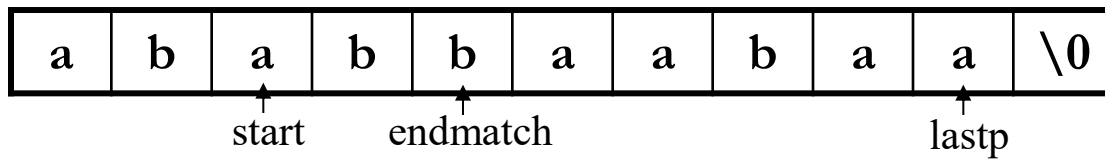
- nfind 시물레이션
 - pat = “aab” 이고 string = “ababbaabaa” 으로 가정
 - string과 pat 배열의 끝을 각각 lasts와 lastp 가리키게 함
 - string[endmatch]와 pat[lastp]를 비교
 - 이 둘이 매치 되면 nfind는 매치 되지 않는 경우가 발생하거나 pat가 모두 매치될때까지 두 스트링 이동을 위해 i, j 사용
 - 변수 start는 매치 되지 않을 경우 i를 재설정하기 위해 사용



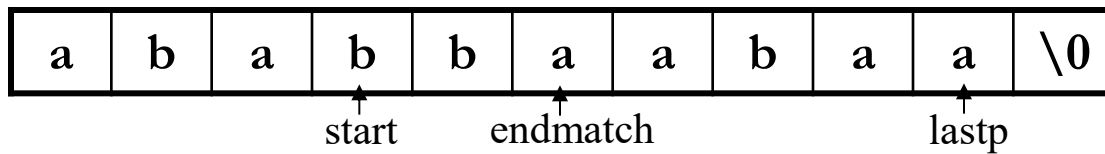
Pattern Matching(4/7)



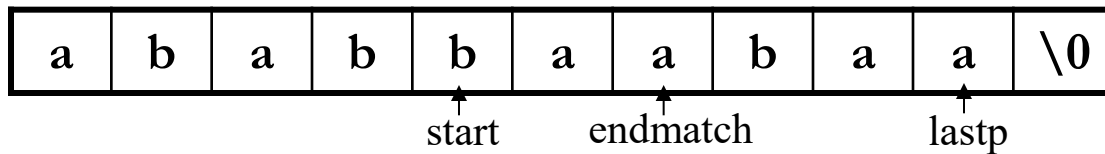
(c) no match



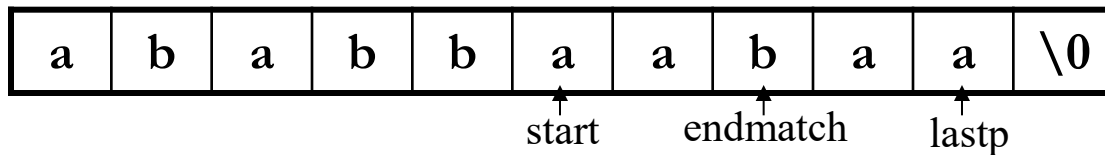
(d) no match



(e) no match



(f) no match



(g) match



Pattern Matching(5/7)

- 실패함수(failure function)

- 정의 : 임의의 패턴 $p=p_0p_1 \dots p_{n-1}$ 이 있을 때 이 패턴의 실패함수, f 는 다음과 같이 정의한다.

$$f(j) = \begin{cases} \text{제일 큰 } i < j, \text{ 여기서 } p_0p_1 \dots p_i = p_{j-i}p_{j-i+1} \dots p_j \text{인 } i \geq 0 \text{이 존재시} \\ -1 \text{ 그 이외의 경우} \end{cases}$$

ex) 패턴 $\text{pat} = \text{abca}b\text{caab}$ 에 대해 f 는 다음과 같다.

j	0	1	2	3	4	5	6	7	8	9
pat	a	b	c	a	b	c	a	c	a	b
f	-1	-1	-1	0	1	2	3	-1	0	1



Pattern Matching(6/7)

- Knuth, Morris, Pratt pattern matching algorithm

```
int pmatch(char *string, char *pat)
{
    /* Knuth, Morris, Pratt의 스트링 매칭 알고리즘 */
    int i = 0, j = 0;
    int lens = strlen(string);
    int lenp = strlen(pat);
    while (i < lens && j < lenp) {
        if (string[i] == pat[j]) {
            i++; j++;
        }
        else if (j == 0) i++;
        else j = failure[j-1]+1;
    }
    return ((j == lenp) ? (i - lenp) : -1);
}
```

*What is the time complexity
of this algorithm?*



Pattern Matching(7/7)

```
void fail(char *pat)
{
    int n = strlen(pat);
    failure[0] = -1;
    for (j=1; j < n; j++) {
        i = failure[j-1];
        while ((pat[j] != pat[i+1]) && (i >= 0)) i = failure[i];
        if (pat[j] == pat[i+1]) failure[j] = i+1;
        else failure[j] = -1;
    }
}
```

*What is the time complexity
of this algorithm?*



Homework II

- Read Chapter 3
- 연습문제 : 1, 3, 5, 7

